SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA010**

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

I T Bhupal/192425243 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: bhupalreddy

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	**100%**	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	**Medical Diagnosis Expert System:** Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and basic clinical information. Represent domain knowledge using production rules and implement both forward and backward chaining to derive conclusions.
2	**Crop Disease Advisory System:** Develop an expert system that identifies possible crop diseases based on leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. Construct suitable production rules and demonstrate diagnosis using forward and backward chaining.
3	**Automobile Fault Diagnosis System:** Design an expert system that identifies probable vehicle faults based on symptoms such as engine noise, starting problems, warning indicators, abnormal vibration, and reduced mileage. Implement the knowledge base using production rules and compare forward and backward reasoning.
4	**Student Academic Advisory System:** Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.
5	**Cybersecurity Threat Diagnosis System:** Construct a rule-based expert system that identifies possible cybersecurity threats from events such as repeated login failures, unusual network traffic, suspicious file activity, and unauthorized access attempts. Implement production rules and demonstrate how forward and backward chaining arrive at a threat classification.

Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

Crop Disease Advisory System

1. Introduction

An expert system is an Artificial Intelligence system that uses a knowledge base and inference mechanisms to solve problems that normally require human expertise. This project develops a Crop Disease Advisory Expert System using Prolog.

The system analyzes agricultural information such as leaf symptoms, soil conditions, and weather data. Based on these facts, the system applies production rules to derive conclusions about potential crop diseases (e.g., Late Blight, Powdery Mildew, Root Rot) and suggests remedial actions.

The system demonstrates two important reasoning approaches:

- Forward chaining
- Backward chaining

The project also demonstrates Prolog concepts including facts, rules, variables, unification, backtracking, and logical inference.

2. Problem Statement

Farmers frequently experience massive yield losses due to delayed or inaccurate identification of crop diseases. Manually identifying diseases based on environmental factors and early physical symptoms can be error-prone and time-consuming without an agricultural expert present.

The objective of this project is to develop an intelligent advisory system that analyzes field observations and automatically provides suitable disease diagnoses and treatment recommendations.

Inputs

The main crop attributes considered by the system are:

- Field ID / Name
- Leaf Symptoms
- Weather Conditions
- Humidity Levels
- Soil Conditions

Outputs

The system produces diagnostic conclusions and recommendations such as:

- Late Blight Detected
- Powdery Mildew Detected
- Root Rot Detected
- Apply Fungicide
- Improve Drainage
- Crop is Healthy

3. Objectives

The main objectives of the project are:

1. To develop a rule-based expert system using Prolog.
2. To represent crop and environmental information using facts.
3. To represent diagnostic decision-making knowledge using production rules.
4. To implement forward chaining for data-driven reasoning.
5. To implement backward chaining for goal-driven reasoning.
6. To demonstrate unification and backtracking in Prolog.
7. To generate meaningful agricultural recommendations.
8. To test the system using multiple field scenarios.

4. Domain Analysis

The selected domain is crop disease diagnosis. The expert system considers several environmental and physical indicators.

Attribute	Possible Values	Meaning
Leaf Symptoms	dark_spots, white_powder, wilting, none	Indicates visible damage on the plant leaf
Weather	cool, warm, normal	Current ambient temperature trends
Humidity	high, normal	Moisture levels in the air
Soil Condition	waterlogged, dry, normal	Moisture and drainage state of the soil

The system combines these attributes through logical rules. For example, when a field has cool weather, high humidity, and dark spots on leaves, the system derives the conclusion that the crop is suffering from Late Blight.

5. Knowledge Acquisition

The knowledge represented in the system consists of agricultural diagnostic conditions and their corresponding recommendations. The knowledge was converted into Prolog facts and rules.

For example, a field's observation information can be represented as:

field(roopesh_farm, dark_spots, cool, high, normal).

This fact represents:

- Field: roopesh_farm
- Leaf Symptom: dark_spots
- Weather: cool
- Humidity: high
- Soil: normal

6. Knowledge Representation

Prolog represents the knowledge using facts, predicates, variables, and rules.

6.1 Crop Facts

A crop fact follows the structure:

field(Name, Leaf, Weather, Humidity, Soil).

Example:

field(roopesh_farm, dark_spots, cool, high, normal).

Other fields are also represented in the knowledge base to test different environmental situations.

6.2 Production Rules

The expert system uses rules to derive conclusions from field facts. Examples of the implemented reasoning are:

Prolog

```
disease_late_blight(Name) :-
    field(Name, dark_spots, cool, high, _).
```

A root rot condition is represented using wilting leaves and waterlogged soil:

Prolog

```
disease_root_rot(Name) :-
    field(Name, wilting, _, _, waterlogged).
```

A specific disease condition produces a treatment recommendation:

Prolog

```
apply_fungicide(Name) :-
    disease_late_blight(Name).
```

7. Inference Design (Forward Chaining)

Forward chaining is a data-driven reasoning approach. The system starts with the known facts and checks which rules can be applied. When the conditions of a rule are satisfied, the corresponding conclusion is derived.

For roopesh_farm, the initial facts are:

- Leaf: dark_spots
- Weather: cool
- Humidity: high

The system fires applicable rules and derives:

- disease_late_blight
- apply_fungicide
- quarantine_crop

8. Backward Chaining

Backward chaining is a goal-driven reasoning approach. Instead of starting from all available facts, the system begins with a goal and checks whether that goal can be proven from the available facts and rules.

For example, the system checks goals such as:

- disease_late_blight -> TRUE
- disease_root_rot -> FALSE
- apply_fungicide -> TRUE

9. Unification and Backtracking

Unification is an important Prolog mechanism that allows variables to match values.

The query ?- disease_late_blight(X). returns fields satisfying the rule:

X = roopesh_farm ;

X = irfan_farm ;

The semicolon allows Prolog to search for additional solutions through backtracking.

10. Procedural and Non-Procedural Paradigm Analysis

Feature	**Procedural**	**Prolog / Declarative**
Main focus	How to solve	What is true
Control flow	Explicit	Mostly handled by Prolog
Knowledge representation	Procedures	Facts and rules
Reasoning	Programmer-defined sequence	Logical inference
Backtracking	Usually implemented manually	Built into Prolog
Unification	Not normally central	Core Prolog mechanism
Suitability for expert systems	More implementation effort	Highly suitable

11. Prolog Implementation

The major components are:

- **Knowledge Base:** Contains field facts.
- **Production Rules:** Determine disease conclusions from the field facts.
- **Forward/Backward Chaining Modules:** Executes reasoning mechanisms.
- **Advisory Module:** Displays final agricultural recommendations.

README.md | implementation.pl U | AT3.png

CO5 > implementation.pl

```
% ============================================================================
% CROP DISEASE ADVISORY EXPERT SYSTEM
% ============================================================================

:- dynamic field/5.

% ============================================================================
% 1. KNOWLEDGE BASE: FACTS (Test Cases)
% Format: field(FarmName, LeafSymptom, Weather, Humidity, SoilCondition).
% ============================================================================

field(roopesh_farm, dark_spots, cool, high, normal).
field(sadhuq_farm, wilting, normal, normal, waterlogged).
field(madhu_farm, white_powder, warm, normal, normal).
field(nivas_farm, none, normal, normal, normal).


% ============================================================================
% 2. PRODUCTION RULES (Disease Identification)
% ============================================================================

% Rule 1: Late Blight
disease(Farm, late_blight) :-
    field(Farm, dark_spots, cool, high, _).

% Rule 2: Root Rot
disease(Farm, root_rot) :-
    field(Farm, wilting, _, _, waterlogged).

% Rule 3: Powdery Mildew
disease(Farm, powdery_mildew) :-
    field(Farm, white_powder, warm, _, _).

% Rule 4: Healthy Crop
disease(Farm, healthy) :-
    field(Farm, none, _, _, _).

```

mysql collage

12. System Architecture

1. Field Observation Facts ->
2. Knowledge Base ->
3. Production Rules ->
4. Forward Chaining / Backward Chaining ->
5. Derived Conclusions ->
6. Final Diagnostic Report ->
7. Treatment Recommendations

13. Algorithm

Forward Chaining Algorithm

1. Accept the field facts.
2. Display the initial facts.
3. Check each production rule.
4. Fire applicable rules.
5. Continue until all applicable rules have been processed.

Backward Chaining Algorithm

1. Select an advisory goal.
2. Search for a rule that can establish the goal.
3. Match the rule conditions with known facts.
4. Return TRUE or FALSE.

14. Testing and Experimental Results

Field Name	**Main Agricultural Situation**	**Result**
Roopesh Farm	Dark spots, cool weather, high humidity	Late Blight, apply fungicide
Sadhuq Farm	Wilting leaves, waterlogged soil	Root Rot, improve soil drainage
Madhu Farm	White powder on leaves, warm weather	Powdery Mildew, apply sulfur spray
Nivas Farm	No symptoms, normal weather, normal soil	Healthy crop, continue maintenance

15. Test Case: Roopesh Farm

Query: ?- diagnose(roopesh_farm).

Initial Facts: Leaf: dark_spots, Weather: cool, Humidity: high

Derived Conclusions:

-> disease_late_blight

-> apply_fungicide

16. Results and Analysis

The experimental results show that the expert system successfully applies its production rules to different field scenarios. For Sadhuq Farm, the combination of wilting leaves and waterlogged soil successfully triggered the root rot diagnosis, skipping irrelevant rules like powdery mildew.

17. Unification and Backtracking Results

The query: ?- disease_late_blight(X). successfully returned multiple matching fields, demonstrating variable matching, unification, multiple solutions, and Prolog backtracking.

18. Limitations

1. The academic categories are represented using simple symbolic values (e.g., high, low).

2. The system does not use numerical data directly (like exact temperature degrees or soil pH).

3. The recommendations depend entirely on predefined rules and cannot learn dynamically.

19. Future Enhancements

1. Integrate IoT sensor data for real-time numerical weather and soil moisture inputs.

2. Develop a computer vision integration using Python and OpenCV to automatically detect leaf symptoms (dark spots, white powder) and feed the symbolic facts into the Prolog inference engine.

3. Add explanation facilities showing exactly why each recommendation was produced.

20. Conclusion

The Crop Disease Advisory Expert System demonstrates how Artificial Intelligence and knowledge-based reasoning can be applied to agriculture. Testing with various field scenarios (Roopesh Farm, Sadhuq Farm, etc.) shows that the system produces distinct, accurate recommendations according to the specific environmental conditions of each field. Overall, the project demonstrates a functional rule-based expert

system capable of providing understandable recommendations through logical reasoning.

21. Screenshots / Evidence

```
==========================================
       CROP DISEASE ADVISORY SYSTEM
==========================================
Farm: roopesh_farm
==========================================
            FORWARD CHAINING
==========================================
Initial Facts
Farm: roopesh_farm
Leaf Symptoms: dark_spots
Weather: cool
Humidity: high
Soil Condition: normal

Rule 1 Fired:
Dark spots + Cool Weather + High Humidity
=> Late Blight Detected

------------------------------------------
Derived Conclusions
------------------------------------------
-> disease_late_blight
-> action_apply_fungicide

==========================================
            BACKWARD CHAINING
==========================================
Goal-driven reasoning:

Goal: disease_late_blight -> true
Goal: disease_root_rot -> false
Goal: disease_powdery_mildew -> false
Goal: healthy_crop -> false
Goal: apply_fungicide -> true
Goal: improve_drainage -> false

Backward chaining completed.
```

```
==========================================
          FINAL ADVISORY REPORT
==========================================
Recommendations:
[*] Apply Fungicide immediately to halt spread.
```

```
========================================
       CROP DISEASE ADVISORY SYSTEM
========================================
Farm: madhu_farm
========================================
            FORWARD CHAINING
========================================
Initial Facts
Farm: madhu_farm
Leaf Symptoms: white_powder
Weather: warm
Humidity: normal
Soil Condition: normal

Rule 3 Fired:
White powder + Warm Weather
=> Powdery Mildew Detected

----------------------------------------
Derived Conclusions
----------------------------------------
-> disease_powdery_mildew
-> action_apply_sulfur_spray

========================================
            BACKWARD CHAINING
========================================
Goal-driven reasoning:

Goal: disease_late_blight -> false
Goal: disease_root_rot -> false
Goal: disease_powdery_mildew -> true
Goal: healthy_crop -> false
Goal: apply_fungicide -> false
Goal: improve_drainage -> false

Backward chaining completed.

========================================
          FINAL ADVISORY REPORT
========================================
Recommendations:
[*] Apply Sulfur Spray to affected leaves.
```

22. References

1. SWI-Prolog Documentation, Prolog language and predicates.

2. Bratko, I., Prolog Programming for Artificial Intelligence.

3. Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach.

23. GitHub Repository

URL: https://github.com/bhupalreddythandlam/MLA0103-AIES/tree/main/CO5/AT1_implementation

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.
7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.
8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.

9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				